Software Engineering 2022

ANS 1 (a)

SEPARATION OF CONCERNS:

In this design concept, we divide the program into different chunks where each chunk addresses a concern.

CONCERN: Any specific feature or behaviour of a sw is 'concern'.

PURPOSE: By dividing the program, we can understand the code more easily. Working and designing gets easier. They are easy to maintain and modify.

MODULARITY:

In the concept of modularity, we break down the system into smaller, self contained units or modules.

Any manageable criteria is acceptable. For instance, we can divide acc to front end / back end, or acc to roles etc.

PURPOSE: Each module focuses on specific part of the system's functionality & can be developed, tested and maintained independently. This increases or enhances code reusability, readability and maintainability.

CASE WHEN 'DIVIDE & CONQUER' STRATEGY IS INAPPROPRIATE:

1. In tightly coupled systems, where components of system are highly interdependent, and changes in one can have cascading effects on other parts
2. In cases, where performance is a critical concern.
3. In cases, where problem is simple & small-scaled. The added complexity of managing modules can outweigh the overall complexity of sw.

IMPACT ON ARGUMENT FOR MODULARITY:

The argument for modularity is that, if you subdivide ~~the~~ sw indefinitely, the effort required to develop it will become negligibly small. But in above cases, the ~~integration~~ effort (cost) associated with integration grows.

Too much modules increases complexity and requires more time to integrate. And too less modules, requires more time to develop or work. Therefore, moderation is required while dividing the module otherwise the organization or integration would cause problems, & hence, the argument for modularity will be inappropriate.

X————————X

Ans 2: (b)

VERIFICATION	VALIDATION
• Verification is about checking if SW meets the specific requirements and design specifications.	• Validation is evaluating final product to ensure it meets intended business needs & requirements.
• To check something with something is verification.	• To check operationally is validation.
• The main question, it answers is: "Are we building the product, right?"	• The main question, it answers is: "Are we building the right product?"
• It focuses on 'process'	• It focuses on 'product'
• It is part of quality control & QA (Quality Assurance) team does it.	• It is part of testing & testing team does validation.

USE OF TEST-CASE DESIGN METHODS & TESTING STRATEGIES:

Yes, both verification & validation processes heavily rely on test-case design methods & testing strategies.

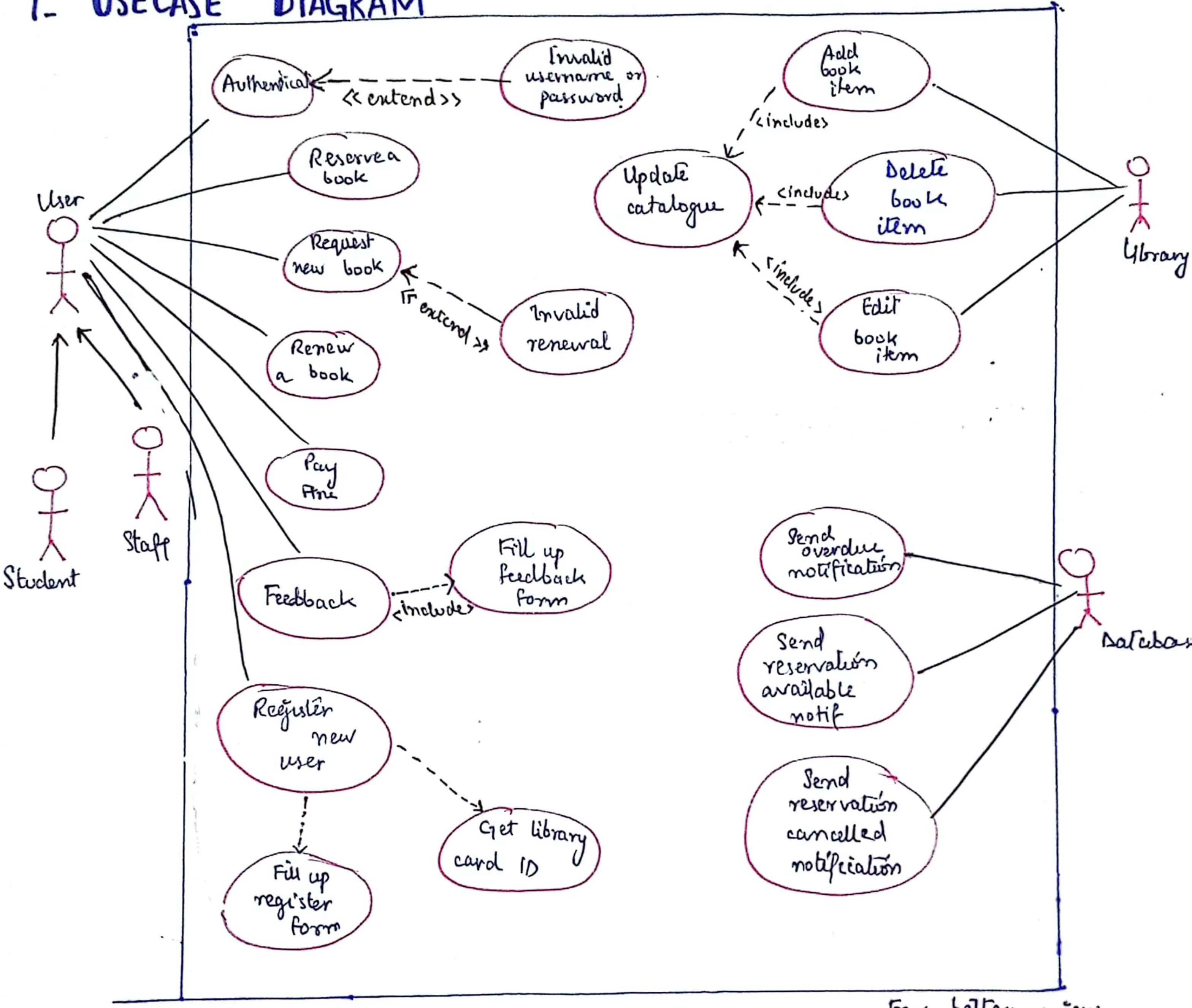
- **Verification** uses techniques like inspection, review & static analysis. It includes unit testing & integration testing.
- **Validation** uses techniques such as acceptance testing (by user), system testing & user testing. Alpha testing & beta testing is the part of validation testing process.

X————————X

pwc ANSWER NO 2 A·F·FERGUSON&CO.

ELEMENTS OF ANALYSIS (a) MODELS :

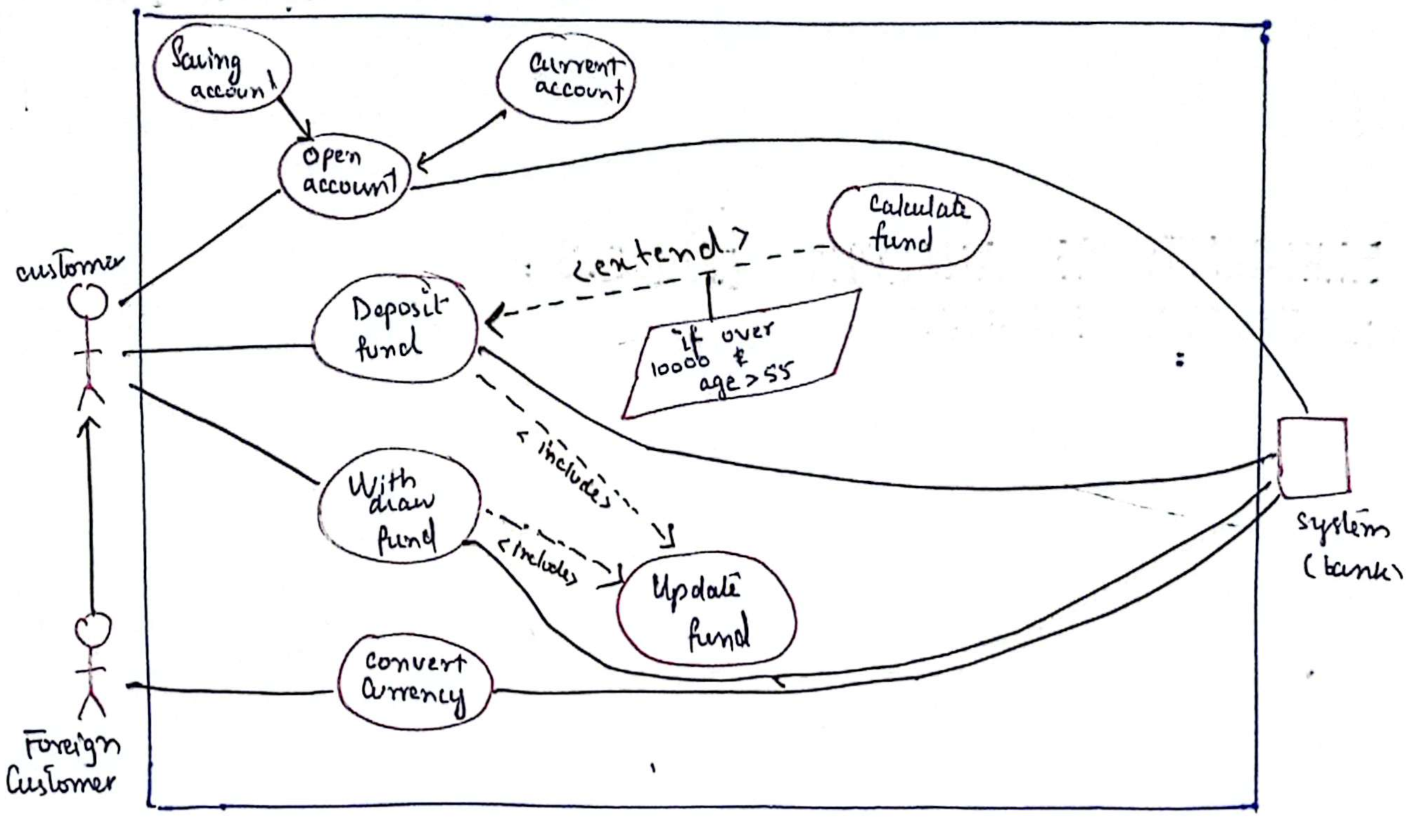
1- USECASE DIAGRAM



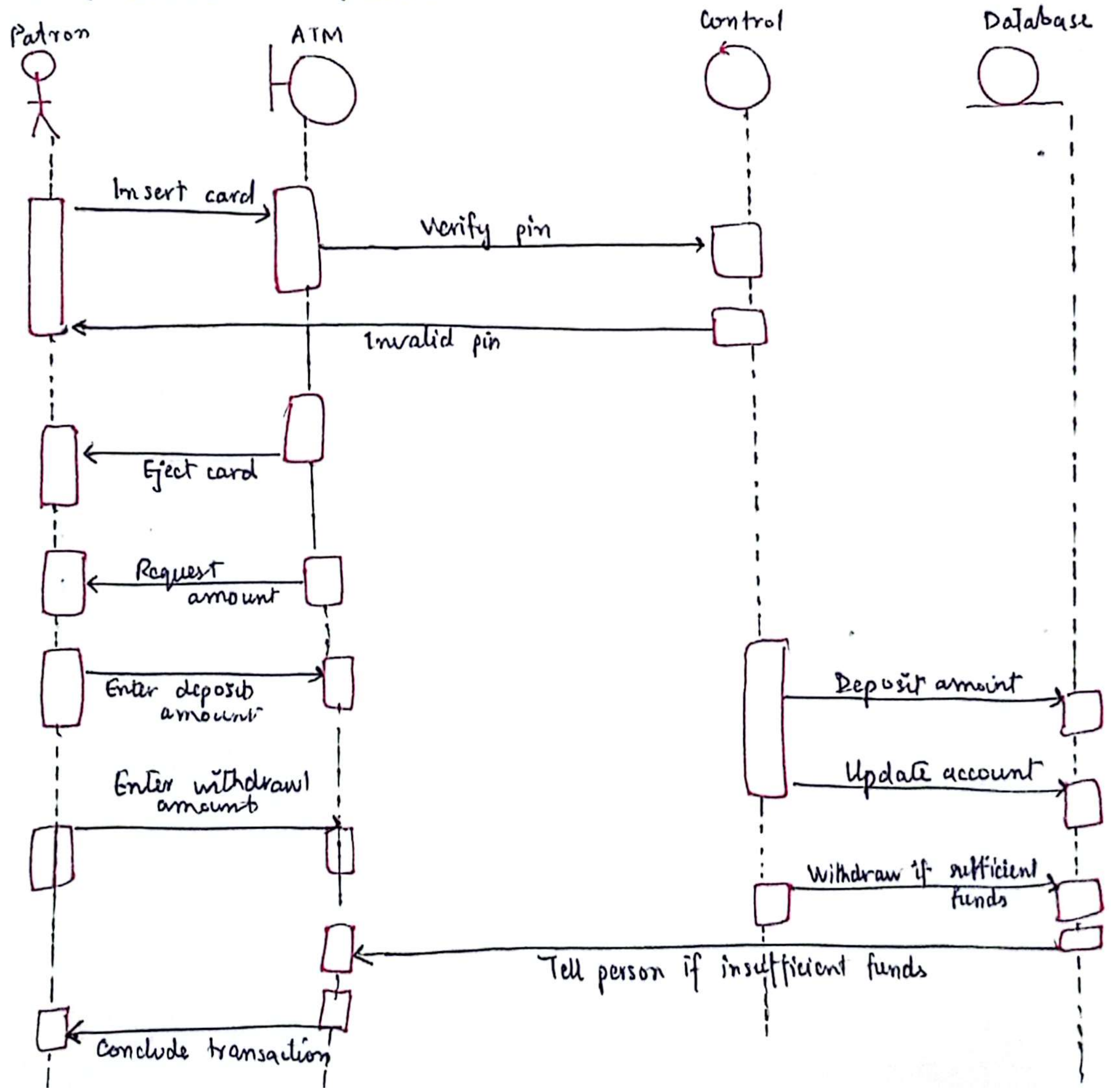
LIBRARY SYSTEM

For better view, view awwabspace.notion.site

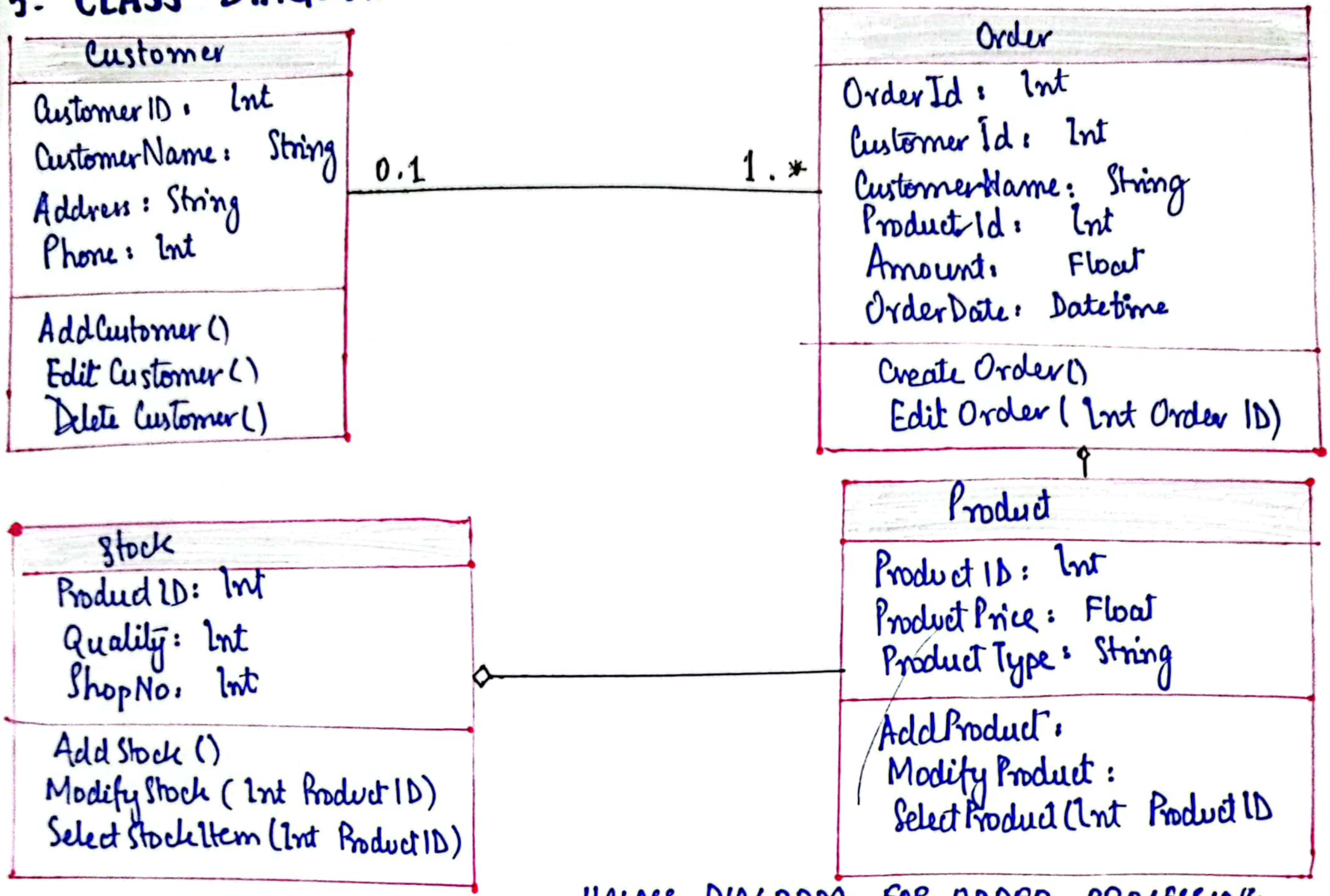
'OR'



2. SEQUENCE DIAGRAM (ATM)

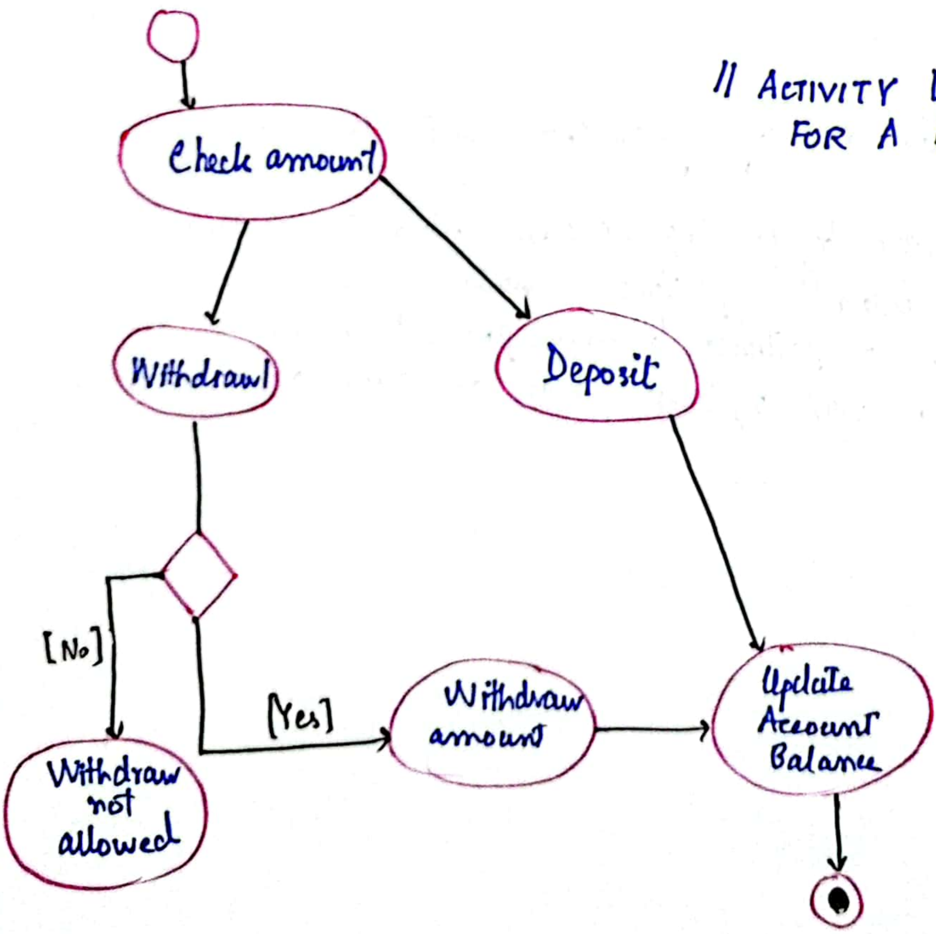


3. CLASS DIAGRAM:



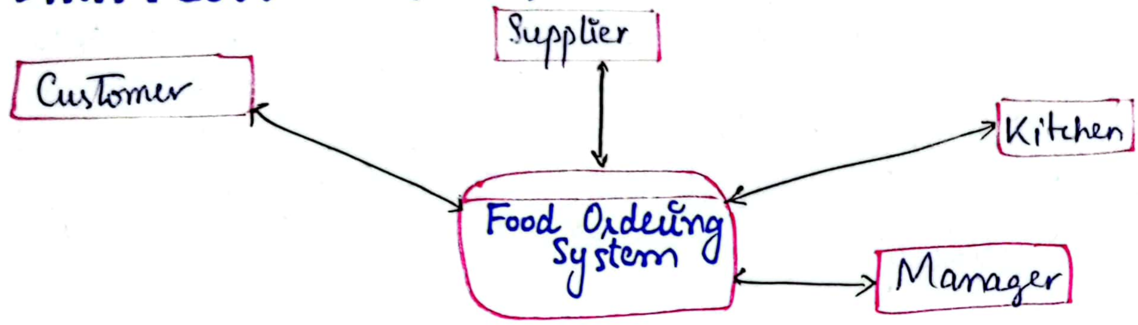
// CLASS DIAGRAM FOR ORDER PROCESSING SYSTEM

4. ACTIVITY DIAGRAM



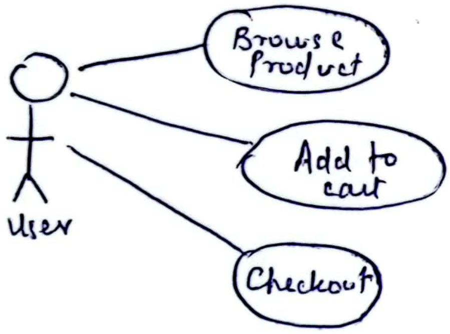
// ACTIVITY DIAGRAM FOR A BANKING SYSTEM

5. DATA FLOW DIAGRAM

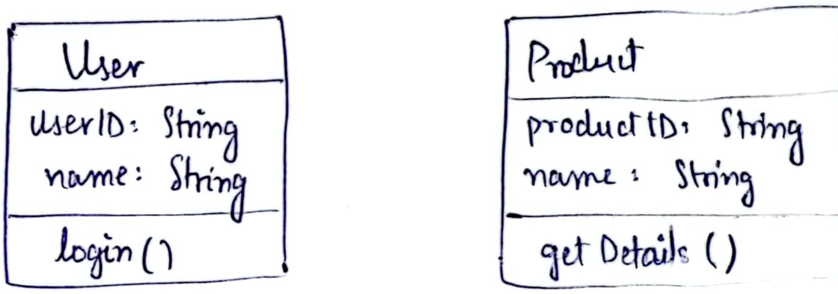


'OR'

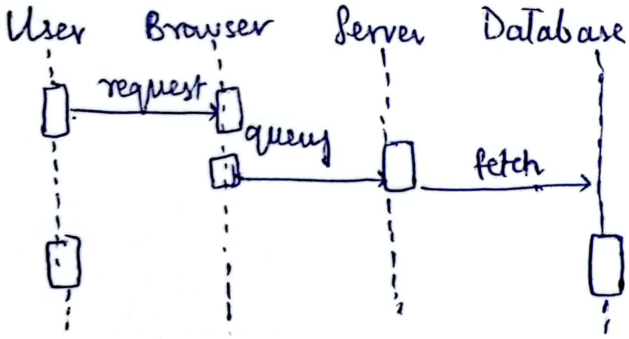
1. USECASE DIAGRAM



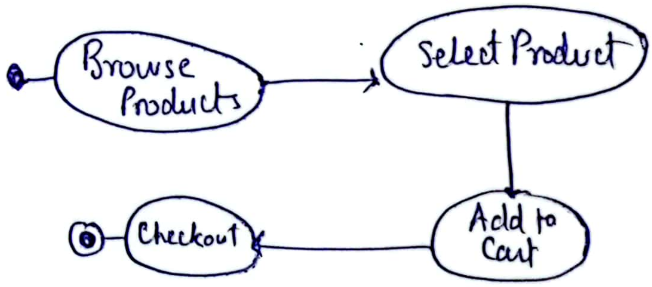
2. CLASS DIAGRAM



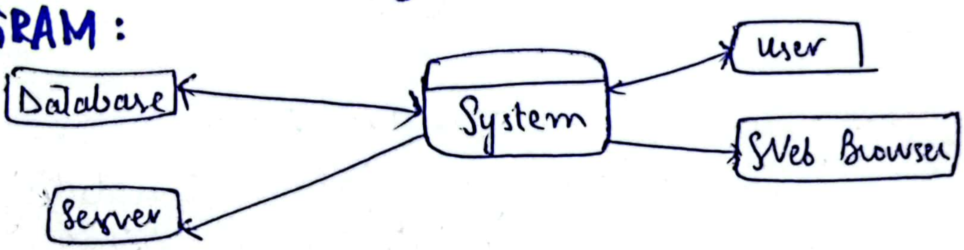
3. SEQUENCE DIAGRAM



4. ACTIVITY DIAGRAM



5. DATA FLOW DIAGRAM :



Note: It is of only '2' marks, so the answer between 'OR' & too lengthy.
This above answer justifies 2 marks.

(b)

Quality goes beyond correctness (i.e., produce right outputs), encompassing aspects like maintainability, efficiency, scalability, readability, reusability, robustness, user experience, reliability, flexibility, & testability. Although, the program is correct in terms of functionality but it can still lack in terms of quality. Quality SW not only produces correct output but also excels in these additional dimensionals

(c)

External failure cost is most expensive, bcz it occurs after the sw product or service has reached the customer or is deployed. It includes warranty claims & repair costs. Fines could be charged due to non-compliance or harm caused by defective products. It may causes damage to company sales, reputation & can affect its market position.

'OR'

External failure cost is the most expensive one cuz resolving issues after delivery/deployment often requires extensive resources for customer support & replacement or repair of products. Defects that reach customer can lead to financial losses, including warranty claims, ~~product~~ & damages to reputation & sales.

GRAVIN'S QUALITY DIMENSIONS:

These are the basis of the Six Sigma which is an International Standard for SW Quality. The eight dimensions are as follows:

1- PERFORMANCE QUALITY:

The primary operating characteristics of a product or service such as speed or accuracy. The features of sw should fulfill their purpose.

2. FEATURE:

Additional features that enhances a product & are used to excite customer. These are not mandatory but are incorporated for user's satisfaction.

3. RELIABILITY:

The product should be able to work without failure over time, through various conditions.

4- CONFORMANCE:

It ensures that the product follows the promised requirements, standards and specifications.

5. DURABILITY:

The lifespan & resilience of product before it deteriorates. It also includes the ability of product to withstand wear & tear.

6- SERVICEABILITY:

The ease & efficiency with which a product can be maintained. It should be able to fix errors and accomodate changes after launch

7. AESTHETICS:

The look & feel of the product. The Better the visual appeal, & design, the more attractive application is.

8- PERCEPTION:

It includes feedback, must get a +ve response. If above 7 dimensions are fulfilled, it automatically comes into play.

ANSWER NO. 3

(a)

AGILE COMMUNICATION & TRADITIONAL SOFTWARE COMMUNICATION:

DIFFERENCES :

FREQUENCY & NATURE OF COMMUNICATION:

- Agile emphasizes continuous communication with frequent, informal meetings, & is often face to face involving all stakeholders.
- Traditional lies on formal, scheduled meetings at specific project milestones. Communication is less frequent & document-driven.

DOCUMENTATON

- Agile focuses on working SW over comprehensive documentation. Documentation is kept minimum & lightweight.
- Traditional requires extensive documentation at every phase of project. (eg SRS, design doc, test plans) etc

STAKEHOLDER INVOLVEMENT

- Agile encourages active & continuous involvement of stakeholders, ensuring feedback is integrated regularly.
- In traditional, they are typically involved at start, end or major milestones.

SIMILARITIES:

- Both aim to deliver projects successfully.
- Both have defined project charter, aim, schedule, budget & a team

ITERATIVE PROCESS & CHANGE MANAGEMENT

An iterative process makes it easier to manage change due to its incremental nature in which changes can easily be accomodated. These processes are flexible & allow quick adaptations to changes and challenges.

AGILE PROCESS & ITERATIVE NATURE:

Yes every agile process is iterative, because it divides project into small, manageable cycles typically called iterations or sprints. It focuses on delivering value incrementally through repeated cycles of development, feedback & refinement. Eg Scrum, XP, Kanban. etc

AGILE & COMPLETION IN ONE ITERATION:

Yes it is theoretically possible if: ~~complete a project in one iteration & still be considered agile if:~~

- The project scope is small enough to be completed wdin 1 iteration.
- The project involves regular feedback, adaptability to changes; & close collaboration with stakeholders.
- The project adheres to agile principles.

However, this scenario is rare & not typical for most agile projects as they are used for complex, ongoing, long-term & real time projects which requires multiple iterations.

X————————X

(b)

Not Included in subject's syllabus or SCOPE...! [GLADLY] !!

ANSWER NO. 4

PROJECT AMENABLE TO PROTOTYPING:

E-COMMERCE WEBSITE

- E-commerce websites heavily rely on user-friendly interfaces.
- Basic functionalities like product listings, shopping carts & checkout processes are well understood, making it easier to create initial prototypes.
- Prototyping enables iterative improvements based on user feedback, & it also helps in identifying potential issues with navigation, layout & user experiences, hence reducing the risk in later stages

APPLICATION DIFFICULT TO PROTOTYPE:

REAL TIME FINANCIAL TRADING PLATFORM:

- They require complex algorithms & real-time data processing which are difficult to ~~prototype~~ simulate in early prototypes. (complex backend logic)
- These platforms must be highly reliable & secure, with requirements that are not easily captured in prototype.
- They often need to integrate with various external financial systems & APIs, making it hard to create a fully functional prototype.

(b)

SOFTWARE REQUIREMENT ENGINEERING TASKS:

SRE encompasses series of tasks & processes aimed at understanding documenting, & managing requirements of SW system throughout its lifecycle. The tasks are as follows:

1. INCEPTION:

It is about basic understanding of the problem. It is the first preliminary communication btw customer & developer to discuss the scope, functionalities, & nature of desired solution. This phase is needed to identify stakeholders, endusers & their viewpoints.

2- ELICITATION:

This phase focuses on gathering the requirements from stakeholders for understanding. Meetings are held btw development team & customer in which multiple problems arises like of problem of scope where the client goes beyond scope, problem of understanding which can be faced at both ends due to language barrier & lastly the problem of volatility, where frequent changes in requirements cause difficulty in leading a project.

TECHNIQUES: Interviews, Surveys, Questionnaire, Brainstorming, Prototyping, Observation & Ethnography are used to collect requirements.

3- ELABORATION

This is the requirement analysis phase, where they are refined & categorized. Experts check & analyze requirements, which are then reviewed. It focuses on developing a refined technical model of SW functions, features & constraints.

4. NEGOTIATION:

The SW engineer reconciles the conflicts b/w what the customer wants & what can be achieved given limited business resources. Requirm are ranked (i.e., prioritized), risk associated with each is identified & analyzed, & the impact of each req on project cost & delivery time is assessed. Mutual consensus & agreement is attained among the multiple stakeholders by settling all the conflicting requirements.

5. SPECIFICATION:

It involves the documenting of requirements in a structured format that is understable to everyone. The output of this phase is SRS (Software Requirement Specification) document which contains clear, concise & unambiguous requirements, functional specifications & usecases.

6. VALIDATION:

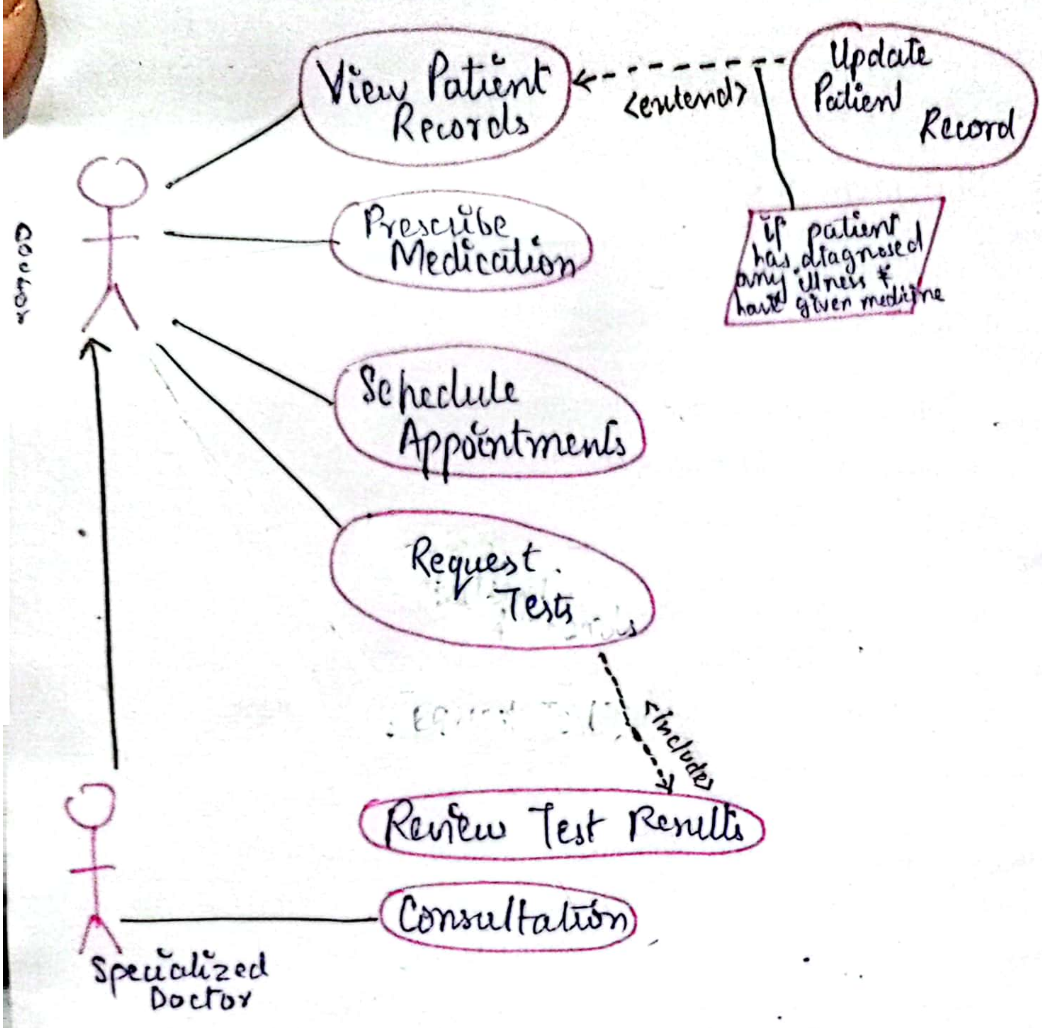
The document thus produced is assessed for quality. It must be reviewed to check the consistency, completeness, realism, & verifiability. The document must conforms to the standards established for process, project & product. It should be ensured that all requirements have been stated unambiguously, & inconsistencies, omissions and errors have been detected & corrected.

7. REQUIREMENT MANAGEMENT:

Change Mgt or Requirement Mgt refers to the ongoing process of managing changes throughout the SW development lifecycle (SDLC), ensuring traceability & alignment with project goals. It involves making changes, addition & deletion of req according to demands of stakeholders.

ANSWER NO. 5

USECASE DIAGRAM FOR HOSPITAL MANAGEMENT:



(b)

TYPES OF RELATIONSHIP IN CLASS DIAGRAM:

1- ASSOSIATION:

It is the connection between two classes, where one class uses the services of other class, where instances of one class can be connected to instances of another class.
A doctor class is associated with patient class

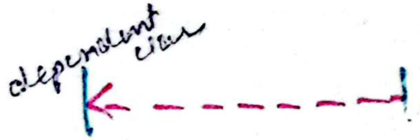
2- INHERITANCE:

The relationship where one class (child/subclass) inherits the attributes & methods of another class (parent/superclass)
Example: Doctor inherits from Staff.

3. REALIZATION:

In Realization, a class implements an interface. It is different from inheritance as it doesn't need parent child relation.
Eg: Doctor is implementing Employee interface.

aiming to develop strategies that stabilize and promote industrial growth amidst global oil market fluctuations.



4. DEPENDENCY :

It is a 'use' relationship, where one class depends on another to function, & it might affect it.

Eg: Billing depends on 'Medical Record'.

5. AGGREGATION :

It is special form of association that represent a whole-part relationship. The part can exist independently of the whole. (weak bonding)

Eg: Hospital aggregates Department

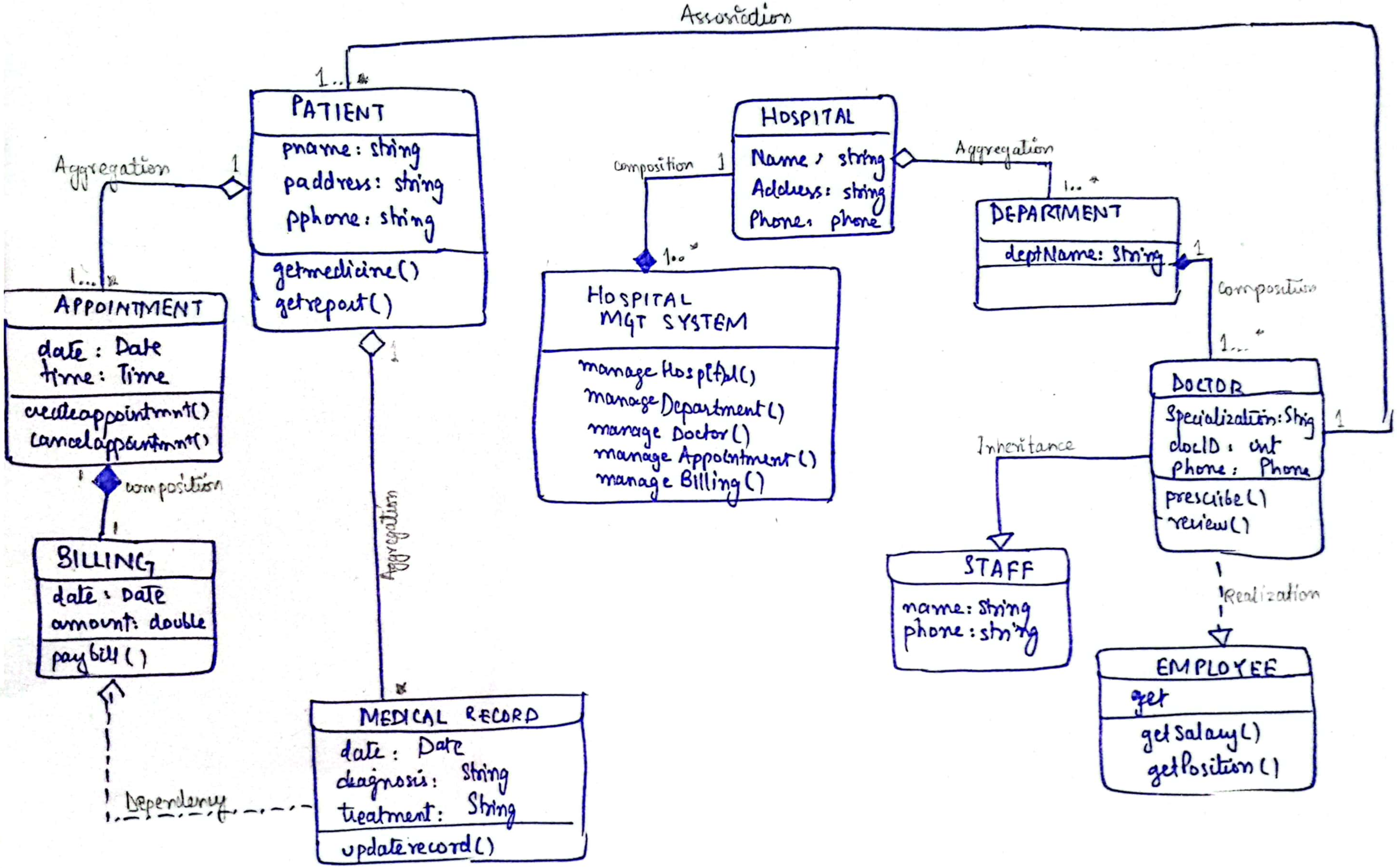
6. COMPOSITION:

Stronger form of aggregation where child classes are dependent on parent class & cannot exist without the whole.

Eg → Hosp mgt system is composed of Hospital mgt ~~system~~.

→ Appointment is composed of Billing as billing can't exist without 'Appointment'.

→ 'Department' is composed of 'Doctors'



Association
1..*
PATIENT
pname: string
paddress: string
pphone: string
getmedicine()
getreport()
Aggregation
1
1..*
APPOINTMENT
date: Date
time: Time
createappointmnt()
cancelappointmnt()
composition
BILLING
date: Date
amount: double
paybill()
Dependency
Aggregation
MEDICAL RECORD
date: Date
diagnosis: String
treatment: String
updaterecord()
HOSPITAL
Name: string
Address: string
Phone: phone
composition
1
1..*
HOSPITAL MGT SYSTEM
manageHospital()
manageDepartment()
manageDoctor()
manageAppointment()
manageBilling()
Aggregation
1..*
DEPARTMENT
deptName: String
1
composition
1..*
DOCTOR
Specialization: String
docID: int
phone: Phone
prescribe()
review()
1
Inheritance
STAFF
name: string
phone: string
Realization
EMPLOYEE
get
getSalary()
getPosition()